People
Innovation
Excellence

Tugas Personal ke-2

Week 7

Data Structures and Algorithm Analysis



2902811940 – RISMANTO

BINUS University Semester Ganjil

2026/2027 Periode 1

BINUS UNIVERSITY ONLINE

People
Innovation
Excellence

Tugas Personal ke-2

Week 7

Linked List & Stack dan Queue

Learning Outcome:

- LO 3: Implement data structures and algorithm in Java.

Topik: Linked List (Operasi Tambah, Hapus, dan Cari Data)

Panduan Pengerjaan

1. Gunakan Java sebagai bahasa pemrograman.
2. Buat class Node dan class LinkedList terpisah.
3. Gunakan Scanner untuk menerima input pengguna.
4. Pastikan setiap operasi (tambah, hapus, cari, tampil) diuji di main().
5. Tampilkan output dengan format yang jelas dan informatif.
6. Tambahkan komentar singkat pada setiap bagian kode penting.

Instruksi Pengumpulan

Mahasiswa wajib menyertakan:

1. Screenshot hasil eksekusi program (menunjukkan program berjalan tanpa error).
2. Link bukti eksekusi program (Replit/GitHub/Google Drive/atau lainnya).
3. Semua bukti disertakan di bagian akhir dokumen tugas sebelum dikumpulkan ke LMS.

Soal Review (Bobot 15 %)

1. Jelaskan konsep dasar Linked List dan perbedaannya dengan Array!
2. Sebutkan kelebihan dan kekurangan Linked List dibanding Array!
3. Jelaskan bagaimana operasi insert dan delete dilakukan pada Linked List!

Soal Case (Bobot 35%

Buatlah **program pengelolaan daftar buku perpustakaan** menggunakan **Single Linked List**. Program ini memiliki menu sebagai berikut:

1. **Tambah Buku (Push)**
   - Input: kodeBuku, judul, dan penulis.
   - Validasi: kodeBuku maksimal 5 karakter, jumlah data minimal 5 buku.
   - Data baru dimasukkan di **akhir daftar**.
2. **Hapus Buku (Pop)**
   - Menghapus **data buku terakhir** dari daftar.
   - Jika daftar kosong, tampilkan pesan *"Tidak ada data untuk dihapus."*
3. **Cari Buku**
   - Input: kodeBuku.
   - Jika ditemukan, tampilkan detail buku.
   - Jika tidak, tampilkan pesan *"Buku tidak ditemukan."*
4. **Tampilkan Semua Buku**
   - Menampilkan seluruh data buku berdasarkan urutan input.
   - Menampilkan total jumlah buku di akhir daftar.
5. **Keluar dari Program**

Contoh Output

```
===== SISTEM DATA BUKU =====
1. Tambah Buku
2. Hapus Buku
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 1

Masukkan Kode Buku: BK101
Masukkan Judul: Struktur Data
Masukkan Penulis: Michael Goodrich
Data berhasil ditambahkan!

Pilih menu: 4
Daftar Buku:
Kode: BK101 | Judul: Struktur Data | Penulis: Michael Goodrich
Total Buku: 1
```

Topik: Stack & Queue

Panduan Pengerjaan

1. Gunakan **dua class**: Queue dan Stack.
2. Gunakan class Node untuk menyimpan data pelanggan (kode, nama, total).
3. Lakukan operasi enqueue-dequeue dan push-display sesuai menu.
4. Gunakan **Scanner** untuk input dan validasi jumlah data ≤ 5.
5. Cetak hasil setiap operasi dengan format yang jelas.
6. Tambahkan komentar di setiap fungsi penting.

People
Innovation
Excellence

Instruksi Pengumpulan

Mahasiswa wajib menyertakan:

1. Screenshot hasil eksekusi program (menunjukkan program berjalan tanpa error).
2. Link bukti eksekusi program (Replit/GitHub/Google Drive/atau lainnya).
3. Semua bukti disertakan di bagian akhir dokumen tugas sebelum dikumpulkan ke LMS.

Soal Review (Bobot 15%)

1. Jelaskan perbedaan antara Stack dan Queue!
2. Sebutkan operasi utama dalam *Stack* dan *Queue*!
3. Berikan contoh penerapan nyata struktur data *Stack* dan *Queue*!

Soal Case (Bobot 35%)

Buatlah program **sistem kasir toko** yang menggunakan **struktur data Stack dan Queue** dengan ketentuan berikut:

Spesifikasi Program

1. **Antrian Pelanggan (Queue Mode)**
    - Input: nomorAntrian, namaPelanggan, totalBelanja.
    - Minimal 5 pelanggan dalam antrian.
    - Jika pelanggan selesai dilayani, lakukan **dequeue**.
2. **Riwayat Transaksi (Stack Mode)**
    - Setiap pelanggan yang selesai dilayani, data transaksinya disimpan ke **stack**.

- Stack menampilkan transaksi terbaru di atas (*LIFO*).

3. **Menu Program**
    - Tambah antrian pelanggan (enqueue).
    - Layani pelanggan (dequeue + push ke stack).
    - Tampilkan antrian saat ini.
    - Lihat riwayat transaksi dari terbaru ke lama.
    - Keluar program.

Contoh Output

```
=== SISTEM KASIR TOKO ===
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 1

Masukkan Nomor Antrian: A001
Masukkan Nama Pelanggan: Andi
Masukkan Total Belanja: 125000
Data pelanggan ditambahkan ke antrian!

Pilih menu: 2
Melayani pelanggan A001 (Andi)
Transaksi disimpan ke riwayat.
```

Jawaban.

Soal Review (Bobot 15 %)

1. **Jelaskan konsep dasar Linked List dan perbedaannya dengan Array!**
2. **Sebutkan kelebihan dan kekurangan Linked List dibanding Array!**
3. **Jelaskan bagaimana operasi insert dan delete dilakukan pada Linked List!**

1. Konsep Dasar Linked List dan Perbedaannya dengan Array

Linked List adalah struktur data yang terdiri dari beberapa node yang saling terhubung menggunakan pointer. Setiap node menyimpan data dan alamat node berikutnya. Berbeda dengan Array yang menggunakan indeks untuk mengakses data secara langsung, Linked List harus menelusuri node satu per satu.

2. Kelebihan dan Kekurangan Linked List dibanding Array

Kelebihan:

- Ukuran data fleksibel dan bisa bertambah atau berkurang.
- Penambahan dan penghapusan data lebih mudah karena tidak perlu menggeser banyak elemen.
- Data tidak harus disimpan secara berurutan di memori.

Kekurangan:

- Pencarian data lebih lambat karena node harus ditelusuri satu per satu.
- Membutuhkan memori tambahan untuk menyimpan pointer.
- Implementasinya lebih rumit dibandingkan Array.

3. Cara Kerja Insert dan Delete pada Linked List

Insert adalah operasi untuk menambahkan node baru dengan mengatur pointer agar node tersebut terhubung ke dalam daftar.

Delete adalah operasi untuk menghapus node dengan mengubah pointer agar node yang dihapus dilewati. Kedua operasi ini bergantung pada posisi node yang ingin ditambahkan atau dihapus.

Program pengelolaan daftar buku perpustakaan menggunakan **Single Linked List**

Source Code LinkedList.Java

```
// 2902811940 - RISMANTO

package linkedlist;



// Implementasi Singly Linked List untuk menyimpan daftar buku.

public class LinkedList {

    private Node head;

    private int size;



    public LinkedList() {

        head = null;

        size = 0;

    }



    public int getSize() {

        return size;

    }
```

```
public boolean isEmpty() {
    return head == null;
}

// Memeriksa apakah kode buku sudah digunakan.
public boolean containsCode(String kodeBuku) {
    Node current = head;
    while (current != null) {
        if (current.kodeBuku.equalsIgnoreCase(kodeBuku)) {
            return true;
        }
        current = current.next;
    }
    return false;
}

// Menambahkan buku di akhir linked list.
public boolean tambahBuku(String kodeBuku, String judul, String penulis) {
    if (containsCode(kodeBuku)) {
        return false;
    }
```

```
    Node newNode = new Node(kodeBuku, judul, penulis);
    if (head == null) {
        head = newNode;
    } else {
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    size++;
    return true;
}

// Menghapus buku terakhir (operasi pop).
public Node hapusBukuTerakhir() {
    if (head == null) {
        return null;
    }
```

```
    Node deleted;
    if (head.next == null) {
        deleted = head;
        head = null;
    } else {
        Node current = head;
        while (current.next.next != null) {
            current = current.next;
        }
        deleted = current.next;
        current.next = null;
    }
    deleted.next = null;
    size--;
    return deleted;
}

// Mencari buku berdasarkan kode.
public Node cariBuku(String kodeBuku) {
    Node current = head;
    while (current != null) {
```

```
            if(current.kodeBuku.equalsIgnoreCase(kodeBuku)) {
                return current;
            }
            current = current.next;
        }
        return null;
    }

    // Menampilkan buku sesuai urutan input beserta jumlahnya.
    public void tampilkanSemuaBuku() {
        if (head == null) {
            System.out.println("Daftar buku masih kosong.");
            System.out.println("Total Buku: 0");
            return;
        }

        System.out.println("\nDaftar Buku:");
        Node current = head;
        while (current != null) {
            System.out.printf("Kode: %s | Judul: %s | Penulis: %s%n",
                    current.kodeBuku, current.judul, current.penulis);
```

```
            current = current.next;
        }
        System.out.println("Total Buku: " + size);
    }
}
```

Source Code Main.java

```
package linkedlist;

import java.util.Scanner;

// Program utama menu pengelolaan daftar buku perpustakaan.
public class Main {
    private static final int MINIMUM_BUKU = 5;

    private static int bacaMenu(Scanner input) {
        while (true) {
            System.out.print("Pilih menu: ");
            String line = input.nextLine().trim();
            try {
```

```
return Integer.parseInt(line);
        } catch (NumberFormatException e) {
            System.out.println("Input harus berupa angka.");
        }
    }
}

private static String bacaTeks(Scanner input, String prompt) {
    while (true) {
        System.out.print(prompt);
        String value = input.nextLine().trim();
        if (!value.isEmpty()) {
            return value;
        }
        System.out.println("Input tidak boleh kosong.");
    }
}

private static boolean tambahBuku(Scanner input, LinkedList daftarBuku) {
    String kode;
    while (true) {
```

```
        kode = bacaTeks(input, "Masukkan Kode Buku (maksimal 5 karakter): ");
        if (kode.length() > 5) {
            System.out.println("Kode buku maksimal 5 karakter. Silakan coba lagi.");
        } else if (daftarBuku.containsCode(kode)) {
            System.out.println("Kode buku sudah digunakan. Masukkan kode lain.");
        } else {
            break;
        }
    }

    String judul = bacaTeks(input, "Masukkan Judul: ");
    String penulis = bacaTeks(input, "Masukkan Penulis: ");

    if (daftarBuku.tambahBuku(kode, judul, penulis)) {
        System.out.println("Data berhasil ditambahkan!");
        System.out.println("Total buku saat ini: " + daftarBuku.getSize());
        return true;
    } else {
        // Pemeriksaan kode ganda juga dilakukan di class LinkedList.
        System.out.println("Data gagal ditambahkan karena kode sudah digunakan.");
        return false;
```

```
    }

}


public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    LinkedList daftarBuku = new LinkedList();
    boolean berjalan = true;
    int totalBukuBerhasilDitambahkan = 0;


    System.out.println("===== PROGRAM KELOLA DAFTAR BUKU =====");
    while (berjalan) {
        System.out.println("\n1. Tambah Buku");
        System.out.println("2. Hapus Buku Terakhir");
        System.out.println("3. Cari Buku");
        System.out.println("4. Lihat Semua Buku");
        System.out.println("5. Keluar");


        int pilihan = bacaMenu(input);
        switch (pilihan) {
            case 1:
                if (tambahBuku(input, daftarBuku)) {
```

```
            totalBukuBerhasilDitambahkan++;
        }
        break;
    case 2:
        Node dihapus = daftarBuku.hapusBukuTerakhir();
        if (dihapus == null) {
            System.out.println("Tidak ada data untuk dihapus.");
        } else {
            System.out.println("Buku terakhir berhasil dihapus: "
                    + dihapus.kodeBuku + " - " + dihapus.judul);
        }
        break;
    case 3:
        String kodeCari = bacaTeks(input, "Masukkan Kode Buku: ");
        Node ditemukan = daftarBuku.cariBuku(kodeCari);
        if (ditemukan != null) {
            System.out.println("Buku ditemukan:");
            System.out.println("Kode: " + ditemukan.kodeBuku);
            System.out.println("Judul: " + ditemukan.judul);
            System.out.println("Penulis: " + ditemukan.penulis);
        } else {
```

```
            System.out.println("Buku tidak ditemukan.");
        }
        break;
    case 4:
        daftarBuku.tampilkanSemuaBuku();
        break;
    case 5:
        if (totalBukuBerhasilDitambahkan < MINIMUM_BUKU) {
            System.out.printf("Belum bisa keluar. Tambahkan minimal %d buku terlebih dahulu. Data berhasil ditambahkan: %d.%n",
                    MINIMUM_BUKU, totalBukuBerhasilDitambahkan);
            break;
        }
        berjalan = false;
        System.out.println("Program selesai. Terima kasih.");
        break;
    default:
        System.out.println("Pilihan tidak tersedia. Pilih menu 1 sampai 5.");
    }
}
input.close();
```

```
    }

}
```

Source Code Node.java

```
package linkedlist;



// Node menyimpan satu data buku dan referensi ke node berikutnya.

public class Node {

    String kodeBuku;

    String judul;

    String penulis;

    Node next;



    public Node(String kodeBuku, String judul, String penulis) {

        this.kodeBuku = kodeBuku;

        this.judul = judul;

        this.penulis = penulis;

        this.next = null;

    }

}
```

Output Tambah Buku

```
===== PROGRAM KELOLA DAFTAR BUKU =====

1. Tambah Buku
2. Hapus Buku Terakhir
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 1
Masukkan Kode Buku (maksimal 5 karakter): BK101
Masukkan Judul: Struktur Data
Masukkan Penulis: Michael Goodrich
Data berhasil ditambahkan!
Total buku saat ini: 1

1. Tambah Buku
2. Hapus Buku Terakhir
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 1
Masukkan Kode Buku (maksimal 5 karakter): BK102
Masukkan Judul: Dilan 1990
Masukkan Penulis: Pidi Baiq
Data berhasil ditambahkan!
Total buku saat ini: 2

1. Tambah Buku
2. Hapus Buku Terakhir
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 1
Masukkan Kode Buku (maksimal 5 karakter): BK103
Masukkan Judul: Tuesday With Morrie
Masukkan Penulis: Mitch Albom
Data berhasil ditambahkan!
Total buku saat ini: 3
```

Output Tampilkan Semua Buku

```
1. Tambah Buku
2. Hapus Buku Terakhir
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 4

Daftar Buku:
Kode: BK101 | Judul: Struktur Data | Penulis: Michael Goodrich
Kode: BK102 | Judul: Dilan 1990 | Penulis: Pidi Baiq
Kode: BK103 | Judul: Tuesday With Morrie | Penulis: Mitch Albom
Total Buku: 3
```

Output Cari Buku

```
1. Tambah Buku
2. Hapus Buku Terakhir
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 3
Masukkan Kode Buku: BK103
Buku ditemukan:
Kode: BK103
Judul: Tuesday With Morrie
Penulis: Mitch Albom
```

Output Hapus Buku Terakhir

```
1. Tambah Buku
2. Hapus Buku Terakhir
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 2
Buku terakhir berhasil dihapus: BK103 - Tuesday With Morrie

1. Tambah Buku
2. Hapus Buku Terakhir
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 4

Daftar Buku:
Kode: BK101 | Judul: Struktur Data | Penulis: Michael Goodrich
Kode: BK102 | Judul: Dilan 1990 | Penulis: Pidi Baiq
Total Buku: 2
```

Soal Review (Bobot 15%)

1. Jelaskan perbedaan antara Stack dan Queue!
2. Sebutkan operasi utama dalam *Stack* dan *Queue*!
3. Berikan contoh penerapan nyata struktur data *Stack* dan *Queue*!

1. Jelaskan perbedaan antara Stack dan Queue!

Stack dan Queue merupakan struktur data yang digunakan untuk mengatur penyimpanan dan pengambilan data. Perbedaan utama keduanya terletak pada urutan data yang diproses.

Stack menggunakan prinsip LIFO (*Last In, First Out*), artinya data yang terakhir masuk akan menjadi data pertama yang keluar. Penambahan dan penghapusan data dilakukan pada bagian atas (*top*).

Sementara itu, Queue menggunakan prinsip FIFO (*First In, First Out*), artinya data yang pertama masuk akan menjadi data pertama yang keluar. Data baru ditambahkan dari bagian belakang (*rear*), sedangkan penghapusan dilakukan dari bagian depan (*front*).

2. Sebutkan operasi utama dalam Stack dan Queue!

Operasi utama pada Stack adalah:

- **Push**, yaitu menambahkan data baru ke bagian atas Stack.
- **Pop**, yaitu menghapus data yang berada di bagian atas Stack.
- **Peek**, yaitu melihat data teratas tanpa menghapusnya.
- **Display**, yaitu menampilkan data yang tersimpan di dalam Stack.

Sementara itu, operasi utama pada Queue adalah:

- **Enqueue**, yaitu menambahkan data baru ke bagian belakang antrian.
- **Dequeue**, yaitu menghapus atau mengambil data yang berada di bagian depan antrian.
- **Peek**, yaitu melihat data paling depan tanpa menghapusnya.

- **Display**, yaitu menampilkan data yang sedang berada dalam antrian.

3. Berikan contoh penerapan nyata struktur data Stack dan Queue!

Contoh penerapan Stack dapat ditemukan pada fitur Undo di aplikasi pengolah teks. Ketika pengguna melakukan beberapa perubahan, sistem dapat membatalkan perubahan terakhir terlebih dahulu. Hal ini sesuai dengan prinsip LIFO.

Sementara itu, Queue dapat diterapkan pada sistem antrian pelanggan di kasir toko. Pelanggan yang datang lebih dahulu akan dilayani lebih dahulu, sedangkan pelanggan yang datang kemudian menunggu giliran. Sistem ini menerapkan prinsip FIFO agar pelayanan berjalan sesuai urutan kedatangan.

Source Code Main.java

```
// 2902811940 - RISMANTO

package kasir;



import java.util.Scanner;



// Program utama sistem kasir berbasis Queue dan Stack.

public class Main {

    private static final int MINIMUM_PELANGGAN_PENGUJIAN = 5;



    private static int bacaMenu(Scanner input) {

        while (true) {

            System.out.print("Pilih menu: ");
```

```
        String line = input.nextLine().trim();
        try {
            return Integer.parseInt(line);
        } catch (NumberFormatException e) {
            System.out.println("Input harus berupa angka.");
        }
    }
}

private static String bacaTeks(Scanner input, String prompt) {
    while (true) {
        System.out.print(prompt);
        String value = input.nextLine().trim();
        if (!value.isEmpty()) {
            return value;
        }
        System.out.println("Input tidak boleh kosong.");
    }
}

private static long bacaTotalBelanja(Scanner input) {
```

```
        while (true) {
            System.out.print("Masukkan Total Belanja: ");
            String line = input.nextLine().trim();
            try {
                long total = Long.parseLong(line);
                if (total < 0) {
                    System.out.println("Total belanja tidak boleh negatif.");
                } else {
                    return total;
                }
            } catch (NumberFormatException e) {
                System.out.println("Total belanja harus berupa bilangan bulat, contoh: 125000.");
            }
        }
    }

    private static void tambahAntrian(Scanner input, Queue antrian) {
        if (antrian.isFull()) {
            System.out.println("Antrian penuh. Maksimal 5 pelanggan dalam antrian.");
            return;
        }
```

```
        String nomor;
        while (true) {
            nomor = bacaTeks(input, "Masukkan Nomor Antrian: ");
            if (antrian.containsNomor(nomor)) {
                System.out.println("Nomor tersebut sudah ada dalam antrian.");
            } else {
                break;
            }
        }
        String nama = bacaTeks(input, "Masukkan Nama Pelanggan: ");
        long total = bacaTotalBelanja(input);
        Node pelanggan = new Node(nomor, nama, total);

        if (antrian.enqueue(pelanggan)) {
            System.out.println("Data pelanggan ditambahkan ke antrian!");
            System.out.println("Jumlah pelanggan dalam antrian: " + antrian.getSize());
            if (antrian.getSize() < MINIMUM_PELANGGAN_PENGUJIAN) {
                System.out.println("Catatan: masukkan hingga 5 pelanggan untuk memenuhi skenario minimum pengujian tugas.");
            }
```

```
    }
    }

    private static void layaniPelanggan(Queue antrian, Stack riwayat) {
        Node pelanggan = antrian.dequeue();
        if (pelanggan == null) {
            System.out.println("Tidak ada pelanggan dalam antrian.");
            return;
        }
        System.out.printf("Melayani pelanggan %s (%s)%n", pelanggan.nomorAntrian, pelanggan.namaPelanggan);
        System.out.printf("Total belanja: Rp%,d%n", pelanggan.totalBelanja);
        riwayat.push(pelanggan);
        System.out.println("Transaksi disimpan ke riwayat.");
    }

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Queue antrian = new Queue();
        Stack riwayat = new Stack();
        boolean berjalan = true;
```

```
int totalPelangganBerhasilDitambahkan = 0;


System.out.println("=== SISTEM KASIR TOKO ===");
while (berjalan) {
    System.out.println("\n1. Tambah Antrian");
    System.out.println("2. Layani Pelanggan");
    System.out.println("3. Tampilkan Antrian");
    System.out.println("4. Lihat Riwayat Transaksi");
    System.out.println("5. Keluar");

    int pilihan = bacaMenu(input);
    switch (pilihan) {
        case 1:
            int jumlahSebelumTambah = antrian.getSize();
            tambahAntrian(input, antrian);
            if (antrian.getSize() > jumlahSebelumTambah) {
                totalPelangganBerhasilDitambahkan++;
            }
            break;
        case 2:
```

```
if (totalPelangganBerhasilDitambahkan < MINIMUM_PELANGGAN_PENGUJIAN) {
        System.out.printf("Masukkan minimal %d pelanggan terlebih dahulu sebelum mulai melayani.%n",
                MINIMUM_PELANGGAN_PENGUJIAN);
    } else {
        layaniPelanggan(antrian, riwayat);
    }
    break;
case 3:
    antrian.tampilkanAntrian();
    break;
case 4:
    riwayat.tampilkanRiwayat();
    break;
case 5:
    berjalan = false;
    System.out.println("Program kasir selesai. Terima kasih.");
    break;
default:
    System.out.println("Pilihan tidak tersedia. Pilih menu 1 sampai 5.");
```

```
        }
        input.close();
    }
}
```

Source Code Node.java

```
package kasir;

// Node menyimpan data pelanggan dan referensi ke node berikutnya.
public class Node {
    String nomorAntrian;
    String namaPelanggan;
    long totalBelanja;
    Node next;

    public Node(String nomorAntrian, String namaPelanggan, long totalBelanja) {
        this.nomorAntrian = nomorAntrian;
        this.namaPelanggan = namaPelanggan;
        this.totalBelanja = totalBelanja;
```

```
        this.next = null;

    }

}
```

Source Code Queue.java

```
package kasir;



// Queue menggunakan prinsip FIFO (First In, First Out).

public class Queue {

    private Node front;

    private Node rear;

    private int size;

    private static final int KAPASITAS_MAKSIMUM = 5;



    public int getSize() {

        return size;

    }



    public boolean isEmpty() {

        return front == null;
```

```
}

public boolean isFull() {
    return size >= KAPASITAS_MAKSIMUM;
}

// Memeriksa nomor antrian yang masih berada dalam antrian.
public boolean containsNomor(String nomor) {
    Node current = front;
    while (current != null) {
        if (current.nomorAntrian.equalsIgnoreCase(nomor)) {
            return true;
        }
        current = current.next;
    }
    return false;
}

// Menambahkan pelanggan ke belakang antrian.
public boolean enqueue(Node pelanggan) {
    if (isFull()) {
```

```
            return false;
        }
        pelanggan.next = null;
        if (rear == null) {
            front = pelanggan;
            rear = pelanggan;
        } else {
            rear.next = pelanggan;
            rear = pelanggan;
        }
        size++;
        return true;
    }

    // Mengambil pelanggan paling depan dan melepaskan link-nya dari antrian.
    public Node dequeue() {
        if (front == null) {
            return null;
        }
        Node pelanggan = front;
        front = front.next;
```

```
        pelanggan.next = null;
        size--;
        if (front == null) {
            rear = null;
        }
        return pelanggan;
    }

    // Menampilkan pelanggan dalam urutan kedatangan.
    public void tampilkanAntrian() {
        if (isEmpty()) {
            System.out.println("Antrian saat ini kosong.");
            return;
        }
        System.out.println("\nAntrian Pelanggan (FIFO):");
        Node current = front;
        int posisi = 1;
        while (current != null) {
            System.out.printf("%d. Nomor: %s | Nama: %s | Total Belanja: Rp%,d%n",
                    posisi, current.nomorAntrian, current.namaPelanggan, current.totalBelanja);
            current = current.next;
```

```
            posisi++;

        }

        System.out.println("Jumlah pelanggan dalam antrian: " + size + "/" +
KAPASITAS_MAKSIMUM);

    }

}
```

Source Code Stack.java

```
package kasir;



// Stack menggunakan prinsip LIFO (Last In, First Out).

public class Stack {

    private Node top;

    private int size;



    public boolean isEmpty() {

        return top == null;

    }



    public int getSize() {
```

```
    return size;
}

// Menyimpan transaksi terbaru di bagian atas stack.
public void push(Node transaksi) {
    transaksi.next = top;
    top = transaksi;
    size++;
}

// Menampilkan transaksi dari yang terbaru ke yang terlama.
public void tampilkanRiwayat() {
    if (isEmpty()) {
        System.out.println("Belum ada riwayat transaksi.");
        return;
    }
    System.out.println("\nRiwayat Transaksi (terbaru ke terlama):");
    Node current = top;
    int nomor = 1;
    while (current != null) {
        System.out.printf("%d. Nomor: %s | Nama: %s | Total Belanja: Rp%,d%n",
```

```
            nomor, current.nomorAntrian, current.namaPelanggan, current.totalBelanja);
            current = current.next;
            nomor++;
        }
        System.out.println("Total transaksi tersimpan: " + size);
    }
}
```

Output Tambah Antrian

```
=== SISTEM KASIR TOKO ===

1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 1
Masukkan Nomor Antrian: A001
Masukkan Nama Pelanggan: Andi
Masukkan Total Belanja: 125000
Data pelanggan ditambahkan ke antrian!
Jumlah pelanggan dalam antrian: 1
Catatan: masukkan hingga 5 pelanggan untuk memenuhi skenario minimum pengujian tugas.

1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 1
Masukkan Nomor Antrian: A002
Masukkan Nama Pelanggan: Budi
Masukkan Total Belanja: 250000
Data pelanggan ditambahkan ke antrian!
Jumlah pelanggan dalam antrian: 2
Catatan: masukkan hingga 5 pelanggan untuk memenuhi skenario minimum pengujian tugas.

1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 1
Masukkan Nomor Antrian: A003
Masukkan Nama Pelanggan: Agus
Masukkan Total Belanja: 300000
Data pelanggan ditambahkan ke antrian!
Jumlah pelanggan dalam antrian: 3
Catatan: masukkan hingga 5 pelanggan untuk memenuhi skenario minimum pengujian tugas.
```

Output Tampilkan Antrian sebelum transaksi

```
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 3

Antrian Pelanggan (FIFO):
1. Nomor: A001 | Nama: Andi | Total Belanja: Rp125.000
2. Nomor: A002 | Nama: Budi | Total Belanja: Rp250.000
3. Nomor: A003 | Nama: Agus | Total Belanja: Rp300.000
4. Nomor: A004 | Nama: Susi | Total Belanja: Rp750.000
5. Nomor: A005 | Nama: Nisa | Total Belanja: Rp1.000.000
Jumlah pelanggan dalam antrian: 5/5
```

Output Layani Pelanggan dan Tampilkan Antrian setelah transaksi

```
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 2
Melayani pelanggan A001 (Andi)
Total belanja: Rp125.000
Transaksi disimpan ke riwayat.

1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 2
Melayani pelanggan A002 (Budi)
Total belanja: Rp250.000
Transaksi disimpan ke riwayat.

1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 3

Antrian Pelanggan (FIFO):
1. Nomor: A003 | Nama: Agus | Total Belanja: Rp300.000
2. Nomor: A004 | Nama: Susi | Total Belanja: Rp750.000
3. Nomor: A005 | Nama: Nisa | Total Belanja: Rp1.000.000
Jumlah pelanggan dalam antrian: 3/5
```

Output Lihat Riwayat Transaksi

```
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 4

Riwayat Transaksi (terbaru ke terlama):
1. Nomor: A002 | Nama: Budi | Total Belanja: Rp250.000
2. Nomor: A001 | Nama: Andi | Total Belanja: Rp125.000
Total transaksi tersimpan: 2
```

Link github: manmaninblack/TP2_2902811940_RISMANTO: Tugas Personal 2 - Data Structures and Algorithm Analysis